

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	M.JYOTHIKA
ROLL NO	AV.SC.U4CSE25216
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5-7
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	8-10
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	11-14
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	14-16
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index	17-25

	5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10)Search data in SLL 11)Display or Traverse 12)Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10)Exit	25-33
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	34-39
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	39-47
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion	47-55

	3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	55-61
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	61-68
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	68-78

WEEK – 1

PROGRAM-1(A):

AIM: A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!".

CODE:

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)
n = int(input("Enter countdown number: "))
countdown(n)
```

OUTPUT:

```
Enter countdown number: 5
5
4
3
2
1
Launch!
>>>
```

1(B).AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power) where:

- p = principal growth factor
- n = number of years

CODE:

```
def power(p, n):
    if n == 0:
        return 1
    else:
```

```
    return p * power(p, n - 1)
p = int(input("Enter p: "))
n = int(input("Enter n: "))
result = power(p, n)


print("Answer =", result)
```

OUTPUT:

```
============= RESTART: C:/Users/manda/Desktop/dsa_
Enter p: 5
Enter n: 4
Answer = 625
>
```

1(C).AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(arr, key, index):
    if index == len(arr):
        return False
    if arr[index] == key:
        return True
    return search(arr, key, index + 1)
ids = [101, 102, 103, 104, 105]
key = int(input("Enter employee ID to search: "))
if search(ids, key, 0):
    print("Employee ID found")
else:
    print("Employee ID not found")
```

OUTPUT:

```
============= RESTART: C:/Users/manda/Deskt
Enter employee ID to search: 123
Employee ID not found
>>
```

1(D).AIM: In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
n = int(input("Enter a number: "))
print("Factorial =", factorial(n))
```

OUTPUT:

```
>>
============= RESTART: C:/Users/manda/Deskt
Enter a number: 23
Factorial = 25852016738884976640000
>>
```

1(E).AIM: Write a recursive Python program to print the first n terms of Fibonacci series.

PROGRAM:

```
def fibonacci(a, b, n):
    if n == 0:
        return
    print(a, end=" ")
    fibonacci(b, a + b, n - 1)
n = int(input("Enter number of terms: "))
fibonacci(0, 1, n)
```

OUTPUT:

```
============= RESTART: C:/Users/manda/Desktop
Enter number of terms: 5
0 1 1 2 3
>
```

WEEK 2

2(A).AIM: Write a python program to implement linear search.

PROGRAM:

```
arr = list(map(int, input("Enter array elements separated by spaces: ").split()))


key = int(input("Enter element to search: "))


found = False


for i in range(len(arr)):
    if arr[i] == key:
        print("Element found at position", i + 1)
        found = True
        break


if not found:
    print("Element not found")
```

OUTPUT:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MS
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============= RESTART: C:/Users/manda/Desktop/dsa_python/week1a
Enter array elements separated by spaces: 10 20 30 40 50
Enter element to search: 30
Element found at position 3
```

2(B).AIM: Write a python program to implement binary search with and without sorted array as an input.

PROGRAM:

(i): Binary Search on a Sorted Array

```
arr = list(map(int, input("Enter sorted array elements: ").split()))
```

```
key = int(input("Enter element to search: "))

low = 0
high = len(arr) - 1

found = False

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        print("Element found at position", mid + 1)
        found = True
        break

    elif key < arr[mid]:
        high = mid - 1

    else:
        low = mid + 1

if not found:
    print("Element not found")
```

OUTPUT:

```
============= RESTART: C:/Users/manda/Desktop/dsa_p
Enter sorted array elements: 10 20 30 40 50 60
Enter element to search: 20
Element found at position 2
```

ii): Binary Search on an Unsorted Array

```
arr = list(map(int, input("Enter array elements separated by spaces: ").split()))
```

```
arr.sort()

print("Sorted array:", arr)

key = int(input("Enter element to search: "))

low = 0
high = len(arr) - 1

found = False

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        print("Element found at position", mid + 1, "in the sorted array")
        found = True
        break

    elif key < arr[mid]:
        high = mid - 1

    else:
        low = mid + 1

if not found:
    print("Element not found")
```

OUTPUT:

```
========== RESTART: C:/Users/manda/Desktop/dsa_python/week
Enter array elements separated by spaces: 11 12 13 14 15 16
Sorted array: [11, 12, 13, 14, 15, 16]
Enter element to search: 34
Element not found
>>
```

WEEK 3

3(A).AIM:Write a python program to implement bubble sort where the input elements are taken from the user

PROGRAM:

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr



n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input("Enter element: ")))

bubble_sort(arr)

print("Sorted array:", arr)
```

OUTPUT:

```
Enter number of elements: 5
Enter element: 23
Enter element: 45
Enter element: 67
Enter element: 12
Enter element: 34
Sorted array: [12, 23, 34, 45, 67]
>>
```

3(B).AIM: Write a python program to implement insertion sort where the input elements are taken from the user

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr


n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input("Enter element: ")))

insertion_sort(arr)
```

print("Sorted array:", arr)

OUTPUT:

```
========== RESTART: C:/Users/Manda/Deskt
Enter number of elements: 6
Enter element: 12
Enter element: 34
Enter element: 45
Enter element: 23
Enter element: 66
Enter element: 55
Sorted array: [12, 23, 34, 45, 55, 66]
>>
```

3(C).AIM: Write a python program to implement selection sort where the input elements are taken from the user

PROGRAM:

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr


n = int(input("Enter number of elements: "))

arr = []
```

```
for i in range(n):
    arr.append(int(input("Enter element: ")))

selection_sort(arr)

print("Sorted array:", arr)
```

OUTPUT:

```
---------- RESTART: C:/Users/manda/Desk
Enter number of elements: 6
Enter element: 12
Enter element: 34
Enter element: 22
Enter element: 55
Enter element: 38
Enter element: 11
Sorted array: [11, 12, 22, 34, 38, 55]
>>>
```

WEEK4

4(A).AIM: Write a python program to implement quick sort.

PROGRAM:

```
def quicksort(a, low, high):
    if low < high:
        pivot = a[low]
        i = low + 1
        j = high

        while True:
            while i <= high and a[i] <= pivot:
                i += 1

            while a[j] > pivot:
                j -= 1
```

```
        if i >= j:
            break

        a[i], a[j] = a[j], a[i]

    a[low], a[j] = a[j], a[low]

    quicksort(a, low, j - 1)
    quicksort(a, j + 1, high)


arr = list(map(int, input("Enter numbers to sort: ").split()))

quicksort(arr, 0, len(arr) - 1)

print("Sorted array:", arr)
```

OUTPUT:

```
============= RESTART: C:/Users/manda/Desktop/
Enter numbers to sort: 11 12 34 23 55 62 10
Sorted array: [10, 11, 12, 23, 34, 55, 62]
```

4(B).AIM: Write a python program to implement merge sort where the input elements are taken from the user

PROGRAM:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    merged = []

    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged


arr = list(map(int, input("Enter elements separated by spaces: ").split()))

print("Sorted array:", merge_sort(arr))
```

OUTPUT:

```
Enter elements separated by spaces: 11 23 39 78 65 40 53
Sorted array: [11, 23, 39, 40, 53, 65, 78]
```

WEEK 5

5(A).AIM: Write a python program to implement a Singly Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Sort the data of SLL
10) Search data in SLL
11) Display or Traverse
12) Exit

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input("Enter data: "))
            self.insert_end(data)
        print("Linked List created successfully.")
```

```
# 2. Insert at Beginning
def insert_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    print("Node inserted at beginning.")

# 3. Insert at End
def insert_end(self, data):
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    temp = self.head
    while temp.next is not None:
        temp = temp.next
    temp.next = new_node
# 4. Insert at Index (0-based)
def insert_index(self, data, index):
    if index < 0:
        print("Invalid index.")
        return
    if index == 0:
        self.insert_beginning(data)
        return

    temp = self.head
```

```
    for i in range(index - 1):
        if temp is None:
            print("Invalid index.")
            return
        temp = temp.next

    if temp is None:
        print("Invalid index.")
        return

    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node
    print("Node inserted at index", index)

# 5. Delete by Value
def delete_value(self, value):
    temp = self.head

    if temp is not None and temp.data == value:
        self.head = temp.next
        print("Node deleted.")
        return

    prev = None
    while temp is not None and temp.data != value:
        prev = temp
        temp = temp.next
```

```
    if temp is None:
        print("Value not found.")
        return

    prev.next = temp.next
    print("Node deleted.")

# 6. Delete First Node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return

    self.head = self.head.next
    print("First node deleted.")

# 7. Delete Last Node
def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return

    if self.head.next is None:
        self.head = None
        print("Last node deleted.")
        return

    temp = self.head
    while temp.next.next is not None:
```

```
        temp = temp.next

    temp.next = None
    print("Last node deleted.")

# 8. Count Nodes
def count(self):
    count = 0
    temp = self.head

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# 9. Sort Linked List
def sort(self):
    if self.head is None:
        print("List is empty.")
        return

    current = self.head
    while current is not None:
        index = current.next
        while index is not None:
            if current.data > index.data:
                current.data, index.data = index.data, current.data
            index = index.next
```

```
            current = current.next

        print("Linked List sorted.")

    # 10. Search Data
    def search(self, value):
        temp = self.head
        index = 0

        while temp is not None:
            if temp.data == value:
                print("Value found at index", index)
                return
            temp = temp.next
            index += 1

        print("Value not found.")

    # 11. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head
        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next
```

```
        print("None")

# Main Menu
sll = SinglyLinkedList()

while True:
    print("\n--- SINGLY LINKED LIST MENU ---")
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Sort the Linked List")
    print("10. Search Data")
    print("11. Display / Traverse")
    print("12. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()
    elif choice == 2:
        data = int(input("Enter data: "))
        sll.insert_beginning(data)

    elif choice == 3:
```

```
    data = int(input("Enter data: "))
    sll.insert_end(data)
    print("Node inserted at end.")

elif choice == 4:
    data = int(input("Enter data: "))
    index = int(input("Enter index (starting from 0): "))
    sll.insert_index(data, index)

elif choice == 5:
    value = int(input("Enter value to delete: "))
    sll.delete_value(value)

elif choice == 6:
    sll.delete_first()

elif choice == 7:
    sll.delete_last()

elif choice == 8:
    sll.count()

elif choice == 9:
    sll.sort()

elif choice == 10:
    value = int(input("Enter value to search: "))
    sll.search(value)
```

```
elif choice == 11:
    sll.display()

elif choice == 12:
    print("Exiting program.")
    break

else:
    print("Invalid choice. Please try again.")
```

OUTPUT:

```
Enter data: 56
Linked List created successfully.

--- SINGLY LINKED LIST MENU ---
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Sort the Linked List
10. Search Data
11. Display / Traverse
12. Exit
Enter your choice: 5
Enter value to delete: 67
Node deleted.
```

WEEK 6

AIM: Write a python program to implement a Double Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Display or Traverse

10)Exit

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input("Enter data: "))
            self.insert_end(data)
        print("Linked List created successfully.")

    # 2. Insert at Beginning
    def insert_beginning(self, data):
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node
```

```
    print("Node inserted at beginning.")

# 3. Insert at End
def insert_end(self, data):
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    temp = self.head
    while temp.next is not None:
        temp = temp.next

    temp.next = new_node
    new_node.prev = temp

# 4. Insert at Index (0-based)
def insert_index(self, data, index):
    if index < 0:
        print("Invalid index.")
        return

    if index == 0:
        self.insert_beginning(data)
        return

    temp = self.head
    for i in range(index - 1):
```

```
            if temp is None:
                print("Invalid index.")
                return
            temp = temp.next

        if temp is None:
            print("Invalid index.")
            return

        new_node = Node(data)
        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node
        print("Node inserted at index", index)

    # 5. Delete by Value
    def delete_value(self, value):
        temp = self.head

        while temp is not None and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found.")
            return
```

```
    if temp.prev is not None:
        temp.prev.next = temp.next
    else:
        self.head = temp.next

    if temp.next is not None:
        temp.next.prev = temp.prev

    print("Node deleted.")

# 6. Delete First Node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return

    self.head = self.head.next

    if self.head is not None:
        self.head.prev = None

    print("First node deleted.")

# 7. Delete Last Node
def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return
```

```
    temp = self.head

    if temp.next is None:
        self.head = None
        print("Last node deleted.")
        return

    while temp.next is not None:
        temp = temp.next

    temp.prev.next = None
    print("Last node deleted.")

# 8. Count Number of Nodes
def count(self):
    count = 0
    temp = self.head

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# 9. Display / Traverse
def display(self):
    if self.head is None:
        print("List is empty.")
```

```
            return

        temp = self.head
        print("None", end=" <-> ")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")

# Main Menu
dll = DoublyLinkedList()

while True:
    print("\n--- DOUBLY LINKED LIST MENU ---")
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))
```

```
if choice == 1:
    dll.create()

elif choice == 2:
    data = int(input("Enter data: "))
    dll.insert_beginning(data)

elif choice == 3:
    data = int(input("Enter data: "))
    dll.insert_end(data)
    print("Node inserted at end.")

elif choice == 4:
    data = int(input("Enter data: "))
    index = int(input("Enter index (starting from 0): "))
    dll.insert_index(data, index)

elif choice == 5:
    value = int(input("Enter value to delete: "))
    dll.delete_value(value)

elif choice == 6:
    dll.delete_first()

elif choice == 7:
    dll.delete_last()

elif choice == 8:
    dll.count()
```

```
elif choice == 9:
    dll.display()

elif choice == 10:
    print("Exiting program.")
    break

else:
    print("Invalid choice. Please try again.")
```

OUTPUT:

```
Enter data: 26
Linked List created successfully.

--- DOUBLY LINKED LIST MENU ---
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 55
Node inserted at end.

--- DOUBLY LINKED LIST MENU ---
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 8
```

WEEK7

7(A).AIM: Write a python program to implement stack operations such as push, pop, peek, display using arrays

PROGRAM:

```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, data):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = data

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            print("Deleted value =", self.stack[self.top])
            self.top -= 1

    def peek(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            print("Top element =", self.stack[self.top])
```

```
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            for i in range(self.top, -1, -1):
                print(self.stack[i], end=" ")
            print()


size = int(input("Enter size of stack: "))

s = Stack(size)

while True:
    print("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        s.push(int(input("Enter value: ")))

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()
```

```
    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
-----------------------------------------
Enter size of stack: 6

1.Push 2.Pop 3.Peek 4.Display 5.Exit\
Enter choice: 1
Enter value: 11

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 23

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 45

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 55

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 56

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 89

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 90
Stack Overflow

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
89 56 55 45 23 11
```

7(B).AIM.Write a python program to implement stack operations such as push, pop, peek, display using linked lists.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
```

```
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Deleted value =", self.top.data)
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Top element =", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
            return
```

```
        temp = self.top

        while temp:
            print(temp.data, end=" ")
            temp = temp.next

        print()

s = Stack()

while True:
    print("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        s.push(int(input("Enter value: ")))

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
```

```
        break


    else:
        print("Invalid choice")
```

OUTPUT:

```
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 12

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 13
Invalid choice

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 15
Invalid choice

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 16
Invalid choice

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 17
Invalid choice

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 18
Invalid choice

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
Deleted value = 12

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 17
Invalid choice

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 3
Stack Underflow
```

WEEK 8

8(A). AIM: Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

PROGRAM:

I): Queue Using Array

```
class Queue:
```

```
def __init__(self, size):
    self.size = size
    self.queue = [0] * size
    self.front = -1
    self.rear = -1

def enqueue(self, data):
    if self.rear == self.size - 1:
        print("Queue Overflow")
        return

    if self.front == -1:
        self.front = 0

    self.rear += 1
    self.queue[self.rear] = data
    print(data, "inserted")

def dequeue(self):
    if self.front == -1 or self.front > self.rear:
        print("Queue Underflow")
    else:
        print(self.queue[self.front], "deleted")
        self.front += 1

        if self.front > self.rear:
            self.front = self.rear = -1

def peek(self):
```

```
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print(*self.queue[self.front:self.rear + 1])


size = int(input("Enter queue size: "))

q = Queue(size)

while True:
    print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        q.enqueue(int(input("Enter element: ")))

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()
```

```
    elif choice == 4:
        q.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 34
Invalid choice

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 55
Invalid choice

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 67
Invalid choice

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 34
Invalid choice

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 45
45 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 67
67 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 2
12 deleted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 67
Invalid choice

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 2
45 deleted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 2
67 deleted
```

II): Queue Using Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
```

```
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new = Node(data)

        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new

        print(data, "inserted")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
            return

        print(self.front.data, "deleted")
        self.front = self.front.next

        if self.front is None:
            self.rear = None

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
```

```
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
            return

        temp = self.front

        while temp:
            print(temp.data, end=" ")
            temp = temp.next

        print()


q = Queue()

while True:
    print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        q.enqueue(int(input("Enter element: ")))

    elif choice == 2:
        q.dequeue()
```

```
    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
56 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 45
45 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 89
89 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 90
90 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 34
34 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 77
77 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 2
11 deleted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 77
Invalid choice

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 2
56 deleted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 3
Front element: 45
```

8(B). AIM: Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

PROGRAM:

I): Circular Queue Using Array

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [0] * size
        self.front = -1
```

```
    self.rear = -1

def enqueue(self, data):
    if (self.rear + 1) % self.size == self.front:
        print("Queue Overflow")
        return

    if self.front == -1:
        self.front = self.rear = 0
    else:
        self.rear = (self.rear + 1) % self.size

    self.queue[self.rear] = data
    print(data, "inserted")

def dequeue(self):
    if self.front == -1:
        print("Queue Underflow")
        return

    print(self.queue[self.front], "deleted")

    if self.front == self.rear:
        self.front = self.rear = -1
    else:
        self.front = (self.front + 1) % self.size

def peek(self):
    if self.front == -1:
```

```
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
            return

        i = self.front

        while True:
            print(self.queue[i], end=" ")

            if i == self.rear:
                break

            i = (i + 1) % self.size

        print()


size = int(input("Enter queue size: "))

q = CircularQueue(size)

while True:
    print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")
```

```
choice = int(input("Enter choice: "))

if choice == 1:
    q.enqueue(int(input("Enter element: ")))

elif choice == 2:
    q.dequeue()

elif choice == 3:
    q.peek()

elif choice == 4:
    q.display()

elif choice == 5:
    break

else:
    print("Invalid choice")
```

OUTPUT:

```
Enter queue size: 5

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 12
12 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 44
44 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 34
34 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 66
66 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 45
45 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 89
Queue Overflow

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 2
12 deleted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 3
Front element: 44
```

II): Circular Queue Using Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None
```

```
def enqueue(self, data):
    new = Node(data)

    if self.front is None:
        self.front = self.rear = new
        new.next = new
    else:
        new.next = self.front
        self.rear.next = new
        self.rear = new

    print(data, "inserted")

def dequeue(self):
    if self.front is None:
        print("Queue Underflow")
        return

    print(self.front.data, "deleted")

    if self.front == self.rear:
        self.front = self.rear = None
    else:
        self.front = self.front.next
        self.rear.next = self.front

def peek(self):
    if self.front is None:
```

```
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
            return

        temp = self.front

        while True:
            print(temp.data, end=" ")
            temp = temp.next

            if temp == self.front:
                break

        print()


q = CircularQueue()

while True:
    print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
```

```
            q.enqueue(int(input("Enter element: ")))

        elif choice == 2:
            q.dequeue()

        elif choice == 3:
            q.peek()

        elif choice == 4:
            q.display()

        elif choice == 5:
            break

        else:
            print("Invalid choice")
```

OUTPUT:

```
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 44
44 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 56
56 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 45
45 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 23
23 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 22
22 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 90
90 inserted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 2
44 deleted

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 4
56 45 23 22 90
```

WEEK 9

AIM: Write a python program to implement a Binary Search Tree (BST) and perform the following operations.

1. Insertion

2. Deletion

3. Tree Traversals – in-order, pre-order and post-order

4. Count Leaf Nodes

5. Count Internal Nodes

6. Height of the Tree

PROGRAM:

class Node:

```
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
```

```
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1
```

```
def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

# ================================================
# MAIN PROGRAM
# ================================================
```

```
root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
    print("6. COUNT LEAVES")
    print("7. COUNT INTERNAL NODES")
    print("8. HEIGHT")
    print("9. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
```

```
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()

    elif choice == 5:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 6:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 7:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 8:
        print("Height of BST:", height(root))
    elif choice == 9:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
7. COUNT INTERNAL NODES
8. HEIGHT
9. EXIT
Enter your choice: 1
Enter the data to be inserted: 72
Element inserted successfully.


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. COUNT LEAVES
7. COUNT INTERNAL NODES
8. HEIGHT
9. EXIT
Enter your choice: 2
Preorder Traversal: 12 34 33 28 56 78 72 90


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. COUNT LEAVES
7. COUNT INTERNAL NODES
8. HEIGHT
9. EXIT
Enter your choice: 8
Height of BST: 4
```

WEEK-10:

AIM: Write a menu driven python program to implement an AVL Tree and perform the following operations.

1. Insertion

2. Deletion

3. Tree Traversals

4. Count Leaf Nodes

5. Count Internal Nodes

6. Height of the Tree

PROGRAM:

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0


#AVL TREE Program
def height(root):
    if root is None:
        return -1
    else:
        return root.h


# LL Rotation
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y


# RR Rotation
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
```

```
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# LR Rotation
def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)

# RL Rotation
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)

# Inorder Traversal
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder Traversal
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
```

```
# Postorder Traversal
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# Insert a node into AVL Tree
def insert(root, x):
    if root is None:
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
```

```
        print("Duplicates not allowed")
    root.h = max(height(root.left),height(root.right)) + 1
    return root


# Delete a node from AVL Tree
def delete(root, x):
    # Element not found
    if root is None:
        return None

    # Search in left subtree
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    # Search in right subtree
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
```

```
    # When target found
    else:
        if root.left is None and root.right is None:
            root = None
        elif root.right is None:
            root = root.left
        elif root.left is None:
            root = root.right
        else:
            successor = root.right   # Find inorder successor
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data   #Copy successor data
            root.right = delete(root.right, successor.data)     # Delete the inorder successor

        root.h = max(height(root.left), height(root.right)) + 1
    return root

# --------------------------------
# Main Program
# --------------------------------

root = None
while True:
    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
```

```
print("5. DELETE")
print("6. EXIT")

choice = int(input("Enter your choice: "))
if choice == 1:
    data = int(input("Enter the data to be inserted: "))
    root = insert(root, data)
elif choice == 2:
    print("Preorder Traversal:")
    preorder(root)
    print()
elif choice == 3:
    print("Inorder Traversal:")
    inorder(root)
    print()
elif choice == 4:
    print("Postorder Traversal:")
    postorder(root)
    print()
elif choice == 5:
    data = int(input("Enter the data to be deleted: "))
    root = delete(root, data)
elif choice == 6:
    print("Program terminated.")
    break
else:
    print("Invalid choice!")
```

OUTPUT:

```
8. HEIGHT
9. EXIT
Enter your choice: 1
Enter the data to be inserted: 72
Element inserted successfully.


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. COUNT LEAVES
7. COUNT INTERNAL NODES
8. HEIGHT
9. EXIT
Enter your choice: 2
Preorder Traversal: 12 34 33 28 56 78 72 90


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. COUNT LEAVES
7. COUNT INTERNAL NODES
8. HEIGHT
9. EXIT
Enter your choice: 8
Height of BST: 4
```

WEEK-11:

AIM: Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

1. Insertion of an element into the Heap.
2. Deletion of the root element from the Heap.
3. Display the array representation of the Heap.

Exit the program

PROGRAM:

```
def insert(heap, value, heap_type):
    heap.append(value)
    i = len(heap) - 1
```

```
    while i > 0:
        parent = (i - 1) // 2

        if heap_type == "max" and heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        elif heap_type == "min" and heap[i] < heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

    print("Element inserted successfully.")

def delete_root(heap, heap_type):
    if len(heap) == 0:
        print("Heap is empty.")
        return

    print("Deleted root element:", heap[0])

    heap[0] = heap[-1]
    heap.pop()

    n = len(heap)
    i = 0

    while True:
        target = i
```

```
        left = 2 * i + 1
        right = 2 * i + 2

        if heap_type == "max":
            if left < n and heap[left] > heap[target]:
                target = left
            if right < n and heap[right] > heap[target]:
                target = right
        else:
            if left < n and heap[left] < heap[target]:
                target = left
            if right < n and heap[right] < heap[target]:
                target = right

        if target != i:
            heap[i], heap[target] = heap[target], heap[i]
            i = target
        else:
            break

def display(heap):
    if len(heap) == 0:
        print("Heap is empty.")
    else:
        print("Heap array representation:", heap)

# Main Program
heap = []
heap_type = ""
```

```
while True:
    print("\n--- BINARY HEAP MENU ---")
    print("1. Select Max Heap")
    print("2. Select Min Heap")
    print("3. Insert Element")
    print("4. Delete Root Element")
    print("5. Display Heap")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        heap_type = "max"
        heap = []
        print("Max Heap selected.")

    elif choice == 2:
        heap_type = "min"
        heap = []
        print("Min Heap selected.")

    elif choice == 3:
        if heap_type == "":
            print("Please select Max Heap or Min Heap first.")
        else:
            value = int(input("Enter value to insert: "))
            insert(heap, value, heap_type)
```

```
    elif choice == 4:
        if heap_type == "":
            print("Please select Max Heap or Min Heap first.")
        else:
            delete_root(heap, heap_type)

    elif choice == 5:
        display(heap)

    elif choice == 6:
        print("Exiting program.")
        break

    else:
        print("Invalid choice. Please try again.")
```

output:

```
Element inserted successfully.

--- BINARY HEAP MENU ---
1. Select Max Heap
2. Select Min Heap
3. Insert Element
4. Delete Root Element
5. Display Heap
6. Exit
Enter your choice: 3
Enter value to insert: 90
Element inserted successfully.

--- BINARY HEAP MENU ---
1. Select Max Heap
2. Select Min Heap
3. Insert Element
4. Delete Root Element
5. Display Heap
6. Exit
Enter your choice: 3
Enter value to insert: 22
Element inserted successfully.

--- BINARY HEAP MENU ---
1. Select Max Heap
2. Select Min Heap
3. Insert Element
4. Delete Root Element
5. Display Heap
6. Exit
Enter your choice: 4
Deleted root element: 90

--- BINARY HEAP MENU ---
1. Select Max Heap
2. Select Min Heap
3. Insert Element
4. Delete Root Element
5. Display Heap
6. Exit
Enter your choice: 5
Heap array representation: [78, 66, 34, 13, 22]
```

WEEK-12:

AIM: Write a program to implement the following three collision resolution techniques in a Hash Table:

1. Separate Chaining
2. Linear Probing
3. Quadratic Probing

PROGRAM:

```
# Separate Chaining
def separate_chaining():
    size = int(input("Enter hash table size: "))
    table = [[] for i in range(size)]

    n = int(input("Enter number of elements: "))

    for i in range(n):
        key = int(input("Enter key: "))
        index = key % size
        table[index].append(key)

    print("Hash Table using Separate Chaining:")
    for i in range(size):
        print(i, ":", table[i])



# Linear Probing
def linear_probing():
    size = int(input("Enter hash table size: "))
    table = [None] * size

    n = int(input("Enter number of elements: "))

    for i in range(n):
        key = int(input("Enter key: "))
        index = key % size
        count = 0
```

```
        while table[index] is not None and count < size:
            index = (index + 1) % size
            count += 1

        if count == size:
            print("Hash Table is full. Cannot insert", key)
        else:
            table[index] = key

    print("Hash Table using Linear Probing:")
    for i in range(size):
        print(i, ":", table[i])



# Quadratic Probing
def quadratic_probing():
    size = int(input("Enter hash table size: "))
    table = [None] * size

    n = int(input("Enter number of elements: "))

    for i in range(n):
        key = int(input("Enter key: "))
        index = key % size
        j = 0

        while j < size:
            new_index = (index + j * j) % size
```

```
            if table[new_index] is None:
                table[new_index] = key
                break

            j += 1

        if j == size:
            print("Cannot insert", key, "- no empty slot found")

    print("Hash Table using Quadratic Probing:")
    for i in range(size):
        print(i, ":", table[i])



# Main Menu
while True:
    print("\n--- HASH TABLE MENU ---")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        separate_chaining()

    elif choice == 2:
        linear_probing()
```

```
    elif choice == 3:
        quadratic_probing()

    elif choice == 4:
        print("Exiting program.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
--- HASH TABLE MENU ---
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 1
Enter hash table size: 5
Enter number of elements: 5
Enter key: 11
Enter key: 12
Enter key: 13
Enter key: 14
Enter key: 15
Hash Table using Separate Chaining:
0 : [15]
1 : [11]
2 : [12]
3 : [13]
4 : [14]

--- HASH TABLE MENU ---
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 2
Enter hash table size: 5
Enter number of elements: 5
Enter key: 11
Enter key: 12
Enter key: 13
Enter key: 14
Enter key: 15
Hash Table using Linear Probing:
0 : 15
1 : 11
2 : 12
3 : 13
4 : 14
```